

Graceful Server Shutdown with Signals

<https://gemini.google.com/app/eda94bd40ca83298>

User prompt: I am at the end of phase 1. I am trying to do the last steps or tasks: Step 6: Stress-test / sanity-check before moving on Before declaring Phase 1 done, do these: Concurrent client test: launch multiple client processes (or a multi-threaded client) hammering Put/Get on the same and different keys simultaneously. Verify no crashes, no garbage data. Run with ThreadSanitizer: compile with -fsanitize=thread -g, run the concurrent test, confirm zero data race reports. Basic error handling: what happens if the client requests an empty key? Negative-size value? Decide on behavior (probably: accept it, KV stores are usually permissive) and don't crash. Graceful shutdown: handle SIGINT/SIGTERM and call server->Shutdown() cleanly rather than just killing the process. This matters in Phase 4 when you need nodes to leave the cluster cleanly during testing. you suggested to do this in order: 1. Server CLI Port Argument (src/server.cpp) 2. Graceful Shutdown (src/server.cpp) 3. Basic Error Handling & Permissive Edge Cases 4. Multi-Threaded Concurrent Client Stress Test (tests/test_client_concurrent.cpp) 5. Running with ThreadSanitizer (TSan) I did till 2. graceful shutdown. This is my server.cpp code: #include <grpcpp/grpcpp.h> #include "storage_engine.h" #include "kv.grpc.pb.h" #include <iostream> #include <csignal> std::unique_ptr<grpc::Server> server; void SignalHandler(int signal) { std::cout << "Received signal " << signal << ", shutting down gracefully." << std::endl; if(server) { server->Shutdown(); } } class KVServiceImpl : public kvstore::KeyValueStore::Service { public: grpc::Status Put(grpc::ServerContext* context, const kvstore::PutRequest* request, kvstore::PutResponse* response) override { storage_.Put(request->key(), request->value()); response->set_success(true); return grpc::Status::OK; } grpc::Status Get(grpc::ServerContext* context, const kvstore::GetRequest* request, kvstore::GetResponse* response) override { auto value = storage_.Get(request->key()); if (value.has_value()) { response->set_found(true); response->set_value(*value); } else { response->set_found(false); } return grpc::Status::OK; } grpc::Status Delete(grpc::ServerContext* context, const kvstore::DeleteRequest* request, kvstore::DeleteResponse* response) override { response->set_success(storage_.Delete(request->key())); return grpc::Status::OK; } private: StorageEngine storage_; }; void RunServer(const uint16_t port) { KVServiceImpl service; grpc::ServerBuilder builder; std::string address = "0.0.0.0:" + std::to_string(port); builder.AddListeningPort(address, grpc::InsecureServerCredentials()); builder.RegisterService(&service); server = builder.BuildAndStart(); std::cout << "Server listening on port " << address << std::endl; std::signal(SIGINT, SignalHandler); std::signal(SIGTERM, SignalHandler); server->Wait(); std::cout << "Server shutdown complete." << std::endl; } int main(int argc, char** argv) { uint16_t port = 50051; if (argc > 1) port = static_cast<uint16_t>(std::stoi(argv[1])); RunServer(port); return 0; } I ran server on terminal to check it and pressed ctrl + c. I got this, explain: ./kv_server Server listening on port 0.0.0.0:50051 ^CReceived signal 2, shutting down gracefully. [mutex.cc : 1426] RAW: Potential Mutex deadlock: @ 0x1061d12f4 absl::lts_20250512::DebugOnlyDeadlockCheck() @ 0x1061d1220 absl::lts_20250512::Mutex::Lock() @ 0x105e06a24 absl::lts_20250512::MutexLock::MutexLock() @ 0x105e0258c absl::lts_20250512::MutexLock::MutexLock() @ 0x104d54588 grpc::Server::ShutdownInternal() @ 0x104ccdb28 grpc::ServerInterface::Shutdown() @ 0x104ccd9d4 SignalHandler() @ 0x18cd9b744 _sigtramp @ 0x18cd92128 _pthread_cond_wait @ 0x1061cf59c absl::lts_20250512::synchronization_internal::PthreadWaiter::Wait() @ 0x1061cf20c AbslInternalPerThreadSemWait_lts_20250512 @ 0x1061d0080 absl::lts_20250512::synchronization_internal::PerThreadSem::Wait() @ 0x1061d0024 absl::lts_20250512::Mutex::DecrementSynchSem() @ 0x1061d4588 absl::lts_20250512::CondVar::WaitCommon() @ 0x104d26d18 absl::lts_20250512::CondVar::Wait() @ 0x104d54c70 grpc::Server::Wait() @ 0x104ccdc8c RunServer() @ 0x104ccdf94 main @ 0x18c9d3e00 start [mutex.cc : 1438] RAW: Acquiring absl::Mutex 0x10a38d318 while holding 0x10a38d318; a cycle in the historical lock ordering graph has been observed [mutex.cc : 1439] RAW: Cycle: [mutex.cc : 1453] RAW: mutex@0x10a38d318 stack: @ 0x1061d12f4 absl::lts_20250512::DebugOnlyDeadlockCheck() @ 0x1061d1220 absl::lts_20250512::Mutex::Lock() @ 0x105e06a24 absl::lts_20250512::MutexLock::MutexLock() @ 0x105e0258c absl::lts_20250512::MutexLock::MutexLock() @ 0x104d54588 grpc::Server::ShutdownInternal() @ 0x104ccdb28 grpc::ServerInterface::Shutdown() @ 0x104ccd9d4 SignalHandler() @ 0x18cd9b744 _sigtramp @ 0x18cd92128 _pthread_cond_wait @ 0x1061cf59c absl::lts_20250512::synchronization_internal::PthreadWaiter::Wait() @ 0x1061cf20c AbslInternalPerThreadSemWait_lts_20250512 @ 0x1061d0080 absl::lts_20250512::synchronization_internal::PerThreadSem::Wait() @ 0x1061d0024 absl::lts_20250512::Mutex::DecrementSynchSem() @ 0x1061d4588 absl::lts_20250512::CondVar::WaitCommon() @ 0x104d26d18 absl::lts_20250512::CondVar::Wait() @ 0x104d54c70 grpc::Server::Wait() @ 0x104ccdc8c RunServer() @ 0x104ccdf94 main @ 0x18c9d3e00 start [mutex.cc : 1461] RAW: dying due to potential deadlock zsh: abort ./kv_server Please be accurate, don't hallucinate. Be updated with latest information. explain the problem and its solution in a very beginner friendly manner.

Response: What Happened?

Your server aborted because `server->Shutdown()` was called directly inside the OS signal handler (`SignalHandler`) while the main thread was already sitting inside `server->Wait()`.

gRPC uses Google's Abseil library (`absl::Mutex`) for internal synchronization. Abseil's deadlock-detector caught your process attempting to acquire an internal lock that the thread was already holding inside `Wait()`, flagged a **self-deadlock / lock re-entrancy cycle**, and aborted the process.

Reading the Stack Trace

Looking closely at your stack trace from bottom to top shows the sequence of events:

1. The Normal State:

```
@ 0x104d54c70 grpc::Server::Wait()
@ 0x104d26d18 absl::lts_20250512::CondVar::Wait()
@ 0x18cd92128 _pthread_cond_wait
```

Your main thread called `server->Wait()` and was sleeping, holding internal gRPC mutexes while waiting for incoming RPC calls.

2. The Signal Interrupt:

```
@ 0x18cd9b744 _sigtramp
@ 0x104ccd9d4 SignalHandler()
```

When you pressed `Ctrl + C`, the Operating System sent a `SIGINT` signal. The OS temporarily suspended the main thread right where it was inside `_pthread_cond_wait` and forced it to execute `SignalHandler()`.

3. The Deadlock:

```
@ 0x104d54588 grpc::Server::ShutdownInternal()
@ 0x1061d1220 absl::lts_20250512::Mutex::Lock()
@ 0x1061d12f4 absl::lts_20250512::DebugOnlyDeadlockCheck()
```

Inside `SignalHandler()`, your code called `server->Shutdown()`. `Shutdown()` tried to lock the gRPC server mutex. Because the main thread was already holding that exact mutex inside `Wait()`, Abseil caught the thread trying to lock the same mutex twice!

4. The Abort:

```
[mutex.cc : 1461] RAW: dying due to potential deadlock
zsh: abort      ./kv_server
```

Abseil terminated the process immediately to prevent the server from hanging indefinitely.

Understanding "Async-Signal Safety" in C++

In C and C++, POSIX signal handlers operate under strict rules called **Async-Signal Safety**:

- When an OS signal (like `SIGINT`) arrives, it interrupts whatever instruction the CPU is executing and jumps into your handler function.
- You **must not** perform complex library operations inside a signal handler:
 - Do **not** call `server->Shutdown()` (acquires non-reentrant mutexes).
 - Do **not** call `std::cout` (I/O streaming uses global locks).
 - Do **not** allocate memory with `malloc` or `new`.

The Fix: Decouple Signal Handling from Server Shutdown

Instead of calling gRPC functions or `std::cout` inside `SignalHandler()`, the signal handler should only do **one single task**: set a thread-safe flag (`std::atomic<bool>`).

The main execution loop checks this atomic flag. When it sees that a signal occurred, it calls `server-`

>Shutdown() cleanly from normal main-thread execution context (outside the signal handler) .

Corrected src/server.cpp Code

Replace your current src/server.cpp with this updated, signal-safe implementation:

```
#include <grpcpp/grpcpp.h>
#include "storage_engine.h"
#include "kv.grpc.pb.h"
#include <iostream>
#include <csignal>
#include <atomic>
#include <thread>
#include <chrono>

// Atomic flag to safely pass signal notifications to the main thread
std::atomic<bool> shutdown_requested{false};

void SignalHandler(int signal) {
    // POSIX Signal Safety Rule:
    // ONLY set atomic variables inside a signal handler.
    // Never call std::cout or gRPC functions here!
    shutdown_requested.store(true);
}

class KVServiceImpl : public kvstore::KeyValueStore::Service {
public:
    grpc::Status Put(
        grpc::ServerContext* context,
        const kvstore::PutRequest* request,
        kvstore::PutResponse* response
    ) override {
        storage_.Put(request->key(), request->value());
        response->set_success(true);
        return grpc::Status::OK;
    }

    grpc::Status Get(
        grpc::ServerContext* context,
        const kvstore::GetRequest* request,
        kvstore::GetResponse* response
    ) override {
        auto value = storage_.Get(request->key());
        if (value.has_value()) {
            response->set_found(true);
            response->set_value(*value);
        } else {
            response->set_found(false);
        }
        return grpc::Status::OK;
    }

    grpc::Status Delete(
        grpc::ServerContext* context,
        const kvstore::DeleteRequest* request,
        kvstore::DeleteResponse* response
    ) override {
        response->set_success(storage_.Delete(request->key()));
        return grpc::Status::OK;
    }

private:
    StorageEngine storage_;
};

void RunServer(const uint16_t port) {
```

```

KVServiceImpl service;
grpc::ServerBuilder builder;

std::string address = "0.0.0.0:" + std::to_string(port);

builder.AddListeningPort(address, grpc::InsecureServerCredentials());
builder.RegisterService(&service);

// Build and launch the server
std::unique_ptr<grpc::Server> server = builder.BuildAndStart();
std::cout << "Server listening on port " << address << std::endl;

// Register signal handlers
std::signal(SIGINT, SignalHandler);
std::signal(SIGTERM, SignalHandler);

// Keep the main thread alive, sleeping briefly while waiting for Ctrl+C
while (!shutdown_requested.load()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
}

// Safely execute graceful shutdown from normal thread context
std::cout << "\nReceived signal, shutting down gracefully..." << std::endl;
server->Shutdown();
server->Wait();

std::cout << "[✓] Server shutdown complete." << std::endl;
}

int main(int argc, char** argv) {
    uint16_t port = 50051;

    if (argc > 1) {
        port = static_cast<uint16_t>(std::stoi(argv[1]));
    }

    RunServer(port);
    return 0;
}

```

How to Test and Verify

1. Recompile your server:

```

cd build
make -j4

```

2. Run `./kv_server` in your terminal:

```

./kv_server 50051

```

3. Press `Ctrl + C`. You will now see:

```

Server listening on port 0.0.0.0:50051
^C
Received signal, shutting down gracefully...
[✓] Server shutdown complete.

```

No Abseil deadlock warnings or `zsh: abort` crashes will occur. You are now ready to proceed to **Step 3 (Basic Error Handling)** and **Step 4 (Concurrent Client Stress Test)**!

User prompt: why is there `Wait()` again?? once `shutdown()` is executed the next code won't be run right?

Response: The short answer is: `server->Shutdown()` does NOT pause execution or exit the program. It is a

common misconception to think `Shutdown()` stops code execution. In reality, `server->Shutdown()` is a **non-blocking (asynchronous)** function call.

Here is what is actually happening under the hood and why `server->Wait()` is strictly necessary.

The Difference Between `Shutdown()` and `Wait()`

Function	What it does	Does it block execution?
<code>server->Shutdown()</code>	Sends a message to gRPC: <i>"Stop accepting new client connections, and tell active worker threads to finish up."</i>	No (returns immediately to the next line of code).
<code>server->Wait()</code>	Pauses the main thread and waits until all gRPC background threads have actually finished cleaning up and terminated.	Yes (blocks until gRPC is 100% finished).

What Happens Without `server->Wait()`? (The Crash Scenario)

If you wrote your shutdown code like this without `Wait()`:

```
server->Shutdown();
// Missing server->Wait()!
std::cout << "Server shutdown complete." << std::endl;
```

Here is the disaster that would occur step-by-step:

1. **`server->Shutdown()` runs:** gRPC starts the shutdown process in the background. It returns control to your main thread in less than a millisecond.
2. **Function exits:** The `RunServer()` function reaches its end and returns to `main()`.
3. **Destructors run:** Because `RunServer()` finished, local C++ objects like `KVServiceImpl service` and `StorageEngine storage_` are destroyed immediately from memory.
4. **Crash / Segmentation Fault:** gRPC's background worker threads—which are still in the middle of closing network sockets or finishing an active `Put/Get` request—try to access `storage_` after it has already been destroyed!
 - This results in a **Use-After-Free** memory crash or a **Segmentation Fault**.

The Complete Execution Flow With `Wait()`

When you include both `Shutdown()` and `Wait()`, the shutdown sequence runs safely in order:

```
// 1. Tell gRPC to begin shutting down (non-blocking)
server->Shutdown();

// 2. Pause main thread until gRPC background threads have fully stopped
server->Wait();

// 3. ONLY executed once all gRPC threads are dead and it is safe to clean up memory
std::cout << "[✓] Server shutdown complete." << std::endl;
```

1. **`server->Shutdown()` initiates the server cleanup.**
2. **`server->Wait()` holds the main thread at that exact line until every background thread owned by gRPC has exited.**
3. **The next line** (`std::cout << "[✓] Server shutdown complete."`) is only executed **after** `Wait()` unblocks.
4. **Clean exit:** `RunServer()` finishes, destructors clean up `storage_` safely without any background threads interfering.

User prompt: do you mean shutdown() creates a new thread of its own?

Response: **No, `server->Shutdown()` does NOT create or spawn a new thread.** Instead, it signals gRPC's **existing** internal background worker threads that it is time to stop working.

How gRPC Threads Actually Work

When you launch your gRPC server, gRPC creates an internal **thread pool** (background worker threads) to listen on the network port and process incoming RPC calls (`Put`, `Get`, `Delete`).

When you call `server->Shutdown()` from your main thread or signal handler:

1. **Executes on the Current Thread:** `Shutdown()` runs on whichever thread called it—it does not launch a new thread.
2. **Flips Internal Flags:** It sets internal flags in memory telling gRPC: *"Stop accepting new incoming network connections."*
3. **Wakes Up Existing Background Threads:** It posts notification events to gRPC's **existing** event queues, waking up its background worker threads.
4. **Returns Immediately:** Because it only updates flags and posts notifications, `server->Shutdown()` finishes in less than a millisecond and returns control to the caller.

Why `Wait()` is Needed to Join the Threads

Because `Shutdown()` only *notifies* the background threads, those background worker threads still need time to:

- Finish executing any active `Put/Get` requests currently in progress.
- Close open network sockets cleanly.
- Exit their execution loops and terminate.

`server->Wait()` is the function that puts your main thread to sleep until all of those **pre-existing** background threads have fully terminated and exited.

Once every background thread is dead, `Wait()` unblocks, allowing your main process to safely exit.
